

Rapport technique — Avis et alertes Montréal

1. Qualités et faiblesses du code

Qualités

L'architecture du projet suit une séparation claire entre le frontend React et le backend Express, ce qui facilite la maintenance et l'évolution indépendante des deux couches.

Côté backend, le dossier routes/ regroupe les responsabilités par domaine (avis.js, subscriptions.js, notifications.js), ce qui rend le code facilement navigable.

Les modèles Mongoose dans db/abonnement.js et db/notification.js définissent des schémas stricts avec contraintes d'unicité sur endpoint, ce qui protège l'intégrité des données.

Côté frontend, la logique des notifications push est encapsulée dans le hook personnalisé hooks/useNotifications.js, séparant proprement la logique métier des composants visuels.

Les trois cas limites (permission refusée, déjà bloquée, API indisponible) y sont gérés explicitement.

La gestion des erreurs dans routes/notifications.js supprime automatiquement les abonnements expirés (codes 410 et 404), évitant l'accumulation de données obsolètes en base.

Faiblesses

Le fichier pages/Accueil/Accueil.jsx concentre beaucoup de responsabilités :

- chargement des données,
- gestion du cache,
- filtrage et pagination.

Il pourrait être allégé en extrayant la logique de filtrage dans un hook dédié useFiltre.js. De plus, l'URL du backend est dupliquée dans services/alertes.js et hooks/useNotifications.js. Une variable d'environnement Vite (VITE_BACKEND_URL) centraliserait cette valeur et faciliterait le changement d'environnement.

2. Qualités et faiblesses de l'application

Qualités

- L'expérience utilisateur reproduit fidèlement l'interface de la Ville de Montréal avec une navigation intuitive, des filtres dropdown avec checkboxes, et des chips de filtres actifs.
- La carte Leaflet dans la page de détail offre une localisation visuelle précise des alertes.
- Le mode hors-ligne est fiable grâce au Service Worker qui applique une stratégie Cache First pour les assets statiques et Stale-While-Revalidate pour les données de l'API.
- Les notifications push fonctionnent de bout en bout, incluant la notification in-app lorsque l'application est ouverte.

Faiblesses

Le backend hébergé sur Render en plan gratuit entre en veille après 15 minutes d'inactivité, causant un délai de démarrage de 30 à 60 secondes lors du premier chargement.

L'extraction de l'arrondissement depuis le titre de l'alerte via expression régulière dans services/listes.js est fragile ; si la Ville modifie le format des titres, le filtrage par arrondissement peut échouer silencieusement.

Les notifications push ne fonctionnent pas dans Safari iOS sans que l'application soit préalablement installée sur l'écran d'accueil. Cette restriction est propre à Apple ; sur Android, Chrome supporte les notifications push directement dans le navigateur sans installation préalable.

Le frontend est hébergé sur GitHub Pages en raison de limitations de crédits sur Netlify. GitHub Pages ne supportant pas le routing côté serveur, l'application utilise HashRouter au lieu de BrowserRouter, ce qui ajoute un # dans toutes les URLs (ex: /#/alertes/1). Sur Netlify avec le fichier netlify.toml, ce problème n'existait pas.

3. Propositions d'améliorations priorisées

1. Centraliser l'URL du backend (priorité haute) Remplacer les occurrences de l'URL codée en dur dans `alertes.js` et `useNotifications.js` par une variable d'environnement `VITE_BACKEND_URL`. Cela éliminerait les risques d'erreur lors du passage entre environnements local et production.

2. Extraire la logique de filtrage dans un hook dédié (priorité moyenne) Créer un hook `useFiltre.js` qui regroupe les états et fonctions de filtrage actuellement dans `Accueil.jsx`. Le composant ne s'occuperait que du rendu, améliorant la lisibilité et la testabilité du code.

3. Mettre en cache les réponses du backend et persister les alertes en base de données (priorité moyenne) Ajouter un système de cache côté serveur dans `routes/avis.js` — par exemple avec `node-cache` — pour stocker les données de l'API de la Ville pendant 24 heures. En complément, persister les alertes dans une collection MongoDB `alertes` lors de chaque récupération. Cette double approche réduirait la dépendance à l'API externe, améliorerait les temps de réponse et garantirait un fonctionnement hors-ligne complet même si l'API municipale est indisponible.

4. Migrer vers Netlify (priorité basse) Lorsque les crédits Netlify seront renouvelés, migrer le frontend de GitHub Pages vers Netlify pour utiliser `BrowserRouter` et éliminer le `#` dans les URLs, offrant une meilleure expérience utilisateur et des URLs plus propres.